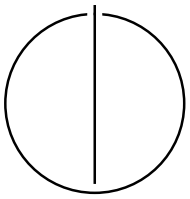
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY - INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Local Time Stepping for the Finite Volume Method in ExaHyPE 2

Raphael Frank



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY - INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Local Time Stepping for the Finite Volume Method in ExaHyPE 2

Lokale Zeitschritte für die Finite Volumen Methode in ExaHyPE 2

Author:	Raphael Frank
Supervisor:	Prof. Dr. Michael Bader
Advisor:	M.Sc. Marc Marot-Lassauzaie
Date:	09.02.2026

I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 09.02.2026 Raphael Frank

Abstract

Hyperbolic partial differential equations (PDEs) often describe important natural phenomena. Examples include wave propagation during tsunamis and the distribution of energy in earthquakes. Accurate forecasts can save lives by enabling timely precautions. Thus, efficient PDE computation is crucial. ExaHyPE 2 is a hyperbolic PDE engine designed to reliably and efficiently solve user-defined PDEs. It provides various solvers, such as Finite Volume (FV) and ADER Discontinuous Galerkin (ADER-DG). Current implementations of these solvers only use global time steps for computing, which forces the entire domain to progress at the rate of the most restrictive cell. However, research suggests that a local time stepping strategy may improve efficiency. In this work, we present an implementation of local time stepping for the Finite Volume method in ExaHyPE. We explain previously proposed strategies for local time stepping and discuss their applicability in ExaHyPE. Furthermore, we describe our method for achieving local time steps and present three approaches for synchronizing neighboring FV cells. We evaluate these approaches against the two existing FV solvers in ExaHyPE and identify scenarios where a local time stepping strategy is generally beneficial. In the evaluation, the tolerance-based synchronization strategy performed best, achieving a 40% speedup over the existing solver using global adaptive time steps. We gain valuable insight into the differences between each implementation. Lastly, we discuss the overall impact on ExaHyPE and outline possible future improvements.

Contents

1 Introduction

Wave systems have a significant impact on people's lives. Naturally occurring events, such as earthquakes and tsunamis, can lead to disastrous outcomes, so a better understanding and accuracy in predicting these events are crucial for taking precautionary actions.

Mathematically, wave systems can be described using hyperbolic partial differential equations (PDEs). Unfortunately, solving PDEs explicitly is often times impossible. Instead, numerical methods are used to approximate a solution.

The problem is that there are significant trade-offs when it comes to solving PDEs numerically. The most prominent one is between accuracy and computation time. Usually, achieving a higher accuracy by decreasing the size of simulated cells requires a massive increase in computation time. With an ever-increasing demand for accuracy in solving PDEs, the computational load becomes immense.

ExaHyPE ("An Exascale Hyperbolic PDE Engine") [1] is an engine to efficiently solve hyperbolic PDEs. It is built on top of the Peano 4 framework [2], which provides the underlying cell traversal and storage schemes. While a lot of previous work has already incorporated powerful and effective features into ExaHyPE, further enhancements in the efficiency of its solvers would potentially reduce computation time.

There are multiple solving techniques in the research when it comes to solving PDE systems. ExaHyPE mainly provides two main solvers: a Finite Volume (FV) solver working with patches [1, 3] and a high-order ADER-DG solver [1, 4]. These solvers each have multiple implementations that differ in the order of accuracy they can compute (i.e., Godunov's and MUSCL-Hancock FV methods) or how the time stepping is performed.

Currently, there are two implementations regarding the time stepping: a version with a fixed global time step and a second that dynamically adjusts the global time step based on the maximum eigenvalues from the previous time step (global adaptive time step).

While a global adaptive time step that utilizes the longest possible stable time step size in every iteration is a clear computational improvement over a fixed time step scheme, research offers another improvement on the solver. The proclaimed strategy uses local time step sizes for each cell in the domain (local time stepping). The premise is to reduce the number of individual time steps required by allowing cells to compute

with their locally longest possible stable time step, rather than the maximum stable time step across the entire domain. This approach would theoretically be more efficient than a global time step in domains where the eigenvalues of the cells differ significantly. That is because only the cells with comparatively large eigenvalues would need to compute with a very small time step, resulting in more iterations for only these cells.

This thesis draws on ideas and approaches from various proposed local time stepping strategies (see chapter 3), adapts them to fit within the ExaHyPE infrastructure, and implements its own solver that utilizes local time steps based on this strategy. We focus on implementing local time steps only for the FV solver rather than the ADER-DG solver. We do so to create a first proof of concept and to explore the possibilities of this approach in general before it can be expanded to other solvers as well.

Since the ExaHyPE engine imposes certain constraints (e.g., cell scheduling) that an implementation from scratch or a theoretical approach might not encounter, it may not be possible to implement all ideas from previous research. Therefore, the goal is to construct our own local time stepping strategy that works well, specifically within ExaHyPE. Our implementation of general local time stepping is explained in detail (see chapter 4). We also present three implemented improvements, covering important aspects and steps that we need to consider.

The different improvements are then evaluated in chapter 5. For this, we simulate four different scenarios with varying configurations for each solver. The results are then analyzed to highlight the pros and cons of our implementations compared to each other as well as the existing FV implementations within ExaHyPE.

The scenarios differ in how the eigenvalues change throughout the simulation, which are used to determine the time step sizes. The computation time for these scenarios is then evaluated for each solver and configuration. This approach not only demonstrates the overall effectiveness of our strategy but also highlights in which scenarios local time stepping would be more efficient than other solver implementations. This comparison may provide users with valuable insight into selecting a solver that works particularly well for their simulations.

Lastly, this thesis summarizes our findings (see chapter 6) to answer our research questions. It evaluates our approach to the implementation of local time stepping in ExaHyPE and provides an outlook on possible ideas that have not been explored or incorporated, as well as plausible goals for future research.

2 Background

Many naturally occurring phenomena that propagate at finite speeds can be expressed as or approximated by a hyperbolic partial differential equation. While a detailed discussion of hyperbolic PDEs goes beyond the purview of this work, we refer the interested reader to the work of Leveque [5].

When performing a simulation, the domain needs to be discretized spatially into data points with a given spacing, as well as chronologically into discrete time steps, since computers do not have unlimited precision or memory. The goal is to find an accurate approximation of the solution that satisfies the conservation laws. This is achieved by simulating the domain at discrete time steps using a numerical solver until a termination time is reached, after which the results can be plotted.

There are different schemes for solving the PDE, with the finite volume (FV) method being one of the first and simplest.

2.1 Finite Volume Scheme

The following basic structure of the FV method was derived from the work of Leveque [5]. In general, the FV method is a solving technique for conservation laws. It partitions the domain into discrete cells and is locally conservative, meaning the numerical flux leaving one cell equals the flux entering its neighbor.

Consider a simple linear advection (transfer of heat/matter along with fluid motion) equation in one dimension, which describes the transport of a quantity q (e.g., pressure) with a constant velocity of $\overline{u} > 0$. The following conservation law holds:

$$\frac{\delta q}{\delta t} + \overline{u}\frac{\delta q}{\delta x} = 0 \tag{2.1}$$

In the finite volume method, the simulation domain is divided into volume cells c_j. Within each cell, the solution is represented as a single constant value q_j that represents the average over that cell:

$$q(x,t) \approx q_j(t) \quad \text{for } x \in [x_{j-\frac{1}{2}}, x_{j+\frac{1}{2}}] \tag{2.2}$$

Because of this discrete approximation, the solution of the domain consists of discon-

tinuous jumps at every border between two cells. Now, integrating the PDE over the cell c_j (to get the desired quantity) results in a semi-discrete form with the change in the cell average depending on the numerical fluxes f at the boundaries:

$$\frac{dq_j}{dt} = -\frac{1}{\Delta x}(f_{j+\frac{1}{2}} - f_{j-\frac{1}{2}}) \tag{2.3}$$

$f_{j+\frac{1}{2}}$ is the numerical flux at the border between cell j and $j+1$ (see Figure 2.1). Here, Equation 2.3 only describes the one-dimensional case. For higher dimensions, we would also have to consider the interfaces in the additional directions (2 per dimension). Because the solutions are represented as constants that are discontinuous at a single point (between cells), our model describes a Riemann problem [5]. The Riemann problem in our example has two initial states for the flux $f_{j+\frac{1}{2}}$: the left state q_j and the right state q_{j+1}. A Riemann solver resolves this discontinuous jump to find a single consistent flux $f_{j+\frac{1}{2}}$. Using this in Equation 2.3, we can calculate the change in quantity in a given cell. When we calculate this change for every cell and set the initial conditions, we can approximate the solution to simulate the domain at discrete time steps.

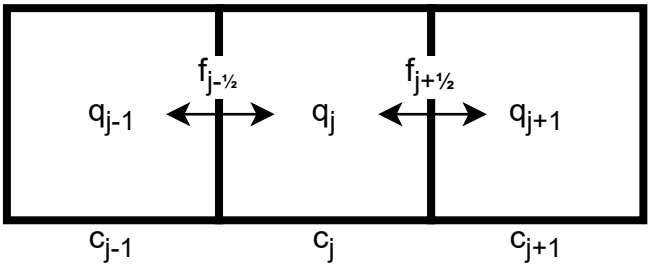


Figure 2.1: The flux f is exchanged between neighboring cells for the Riemann solver to compute the conservative cell quantity q_j.

While the basic version of the FV scheme, Godunov's method, only works with first-order derivatives, there is also an extended version that incorporates second-order derivatives to increase accuracy, called MUSCL-Hancock [6]. However, for simplicity, all simulations in this work will only use Godunov's method.

While there are other numerical schemes that promise higher orders of accuracy or more efficient computation, the FV method has proven itself to be numerically stable and robust for finding accurate solutions to hyperbolic PDEs. The efficiency and stability of the FV method in practice depend heavily on the time steps used and the overall time-stepping strategy.

2.2 Global Time Stepping

As seen in the previous section, solving the integral to calculate the change in quantity dq_j requires specifying the time step dt.

2.2.1 Fixed Time Step Size

The easiest thing to do is to set a fixed time step size at the beginning of the simulation to be used until termination. While this is a rather simple approach, it heavily depends on the choice of the normalized time step size. If chosen too low, the computation time quickly increases and can become unreasonably large. The bigger problem, however, arises when the time step size is chosen too large: the computation becomes too inaccurate, or even worse, unstable, to the point where it crashes. In particular, global fixed time stepping becomes problematic when we need to use very small time steps only occasionally in our simulation (e.g., due to sudden shocks or explosions). We can resolve this problem by dynamically adapting the time step size during the simulation.

An illustration of how the cells progress over time using a fixed time step size Δt is shown in Figure 2.2. For simplicity, we look at a one-dimensional domain with the horizontal axis representing space and the vertical axis representing time, with each time step marked.

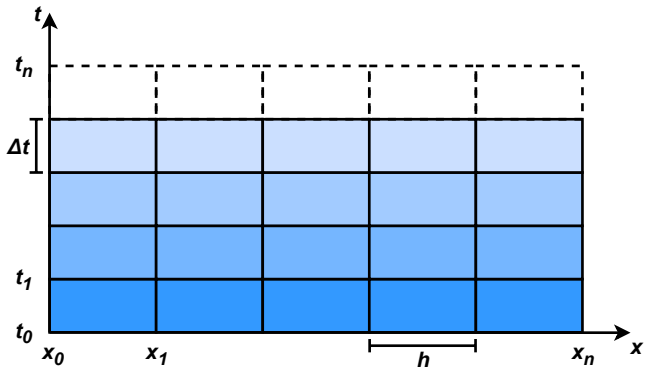


Figure 2.2: One-dimensional domain using a fixed time step size. The horizontal axis represents the different cells in space, while the vertical axis shows how the cells proceed together in time (fading blue).

2.2.2 Adaptive Time Step Size

The goal is to find the maximum possible time step size that keeps the computation accurate and, first of all, stable. The CFL condition, named after Courant, Friedrich, and Lewy, who first described it in [7], states that the computation of a cell is stable as long as the travel speed of the information within each cell is below the size of the cell. This means that the time step size Δt can be calculated by taking the maximum cell size (height, width, etc.) $h_{\max}$ as well as the maximum eigenvalue $\lambda_{\max}$ (i.e., the maximum wave propagation speed) into account. This results in the following equation (see Figure 2.3 for reference):

$$\Delta t \leq \mathrm{CFL}_N \cdot \frac{h_{\max}}{|\lambda_{\max}|} \tag{2.4}$$

CFL_N is a stability factor with $\mathrm{CFL}_N < 1$ that depends on the polynomial order [1, 8]. We can decrease the stability factor for greater stability at the expense of longer computation time.

Using this equation for a stable time step size, we can take the maximum eigenvalue of the entire domain and the maximum cell size to calculate the maximum stable time step size (the admissible time step size). This admissible time step size is then calculated after each iteration and used as the new time step size in the next iteration. This approach guarantees stable, faster computation throughout the simulation compared to fixed-time stepping because it accommodates periods with lower wave propagation, allowing larger time step sizes.

An illustration of how the cells progress over time using an adaptive time step size is shown in Figure 2.3. It can be seen that the time step size changes after each iteration.

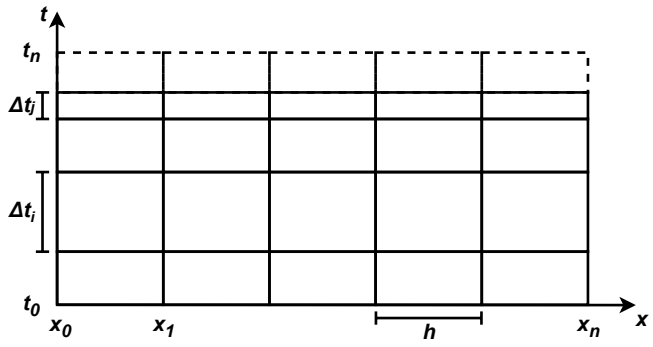


Figure 2.3: One-dimensional domain using an adaptive time step size. Δt_i is larger than Δt_j, which saves computation time since global fixed time stepping would have to use Δt_j for the entire simulation.

2.3 Local Time Stepping

In the previous section, we improved our scheme to include different time steps across the time domain by allowing different time step sizes for different timestamps. We can take this approach even further by allowing different time-step sizes not only across the time domain but also across the space domain. The result is local time stepping, which allows each FV cell to have its own time step size.

After Courant et al. [7], we know that the maximum stable time step size is limited by the propagation speed of the wave (e.g., the maximum eigenvalue). One could easily imagine a scenario in which wave propagation is much faster in some cells and relatively slow in the rest of the domain. One example would be a tsunami in open seas. With global adaptive time stepping, cells where wave propagation is small need to iterate with the same small time step as cells with high wave propagation. This leads to unnecessarily large computation time in these cells (see Figure 2.4).

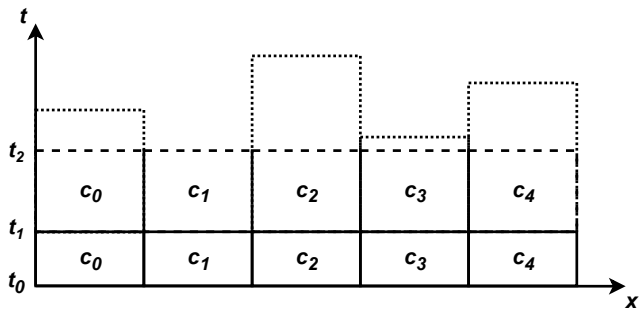


Figure 2.4: Adaptive Time Stepping. The other cells could potentially have a larger time step size (dotted line) after $t = t_1$ but are constrained by cell c_1, which has the largest eigenvalue in the domain.

We can accommodate this by calculating the maximum stable time step size (see Equation 2.4) not only for the entire domain but also for each cell individually. This way, cells with very small wave propagation save computation time by using larger time step sizes, leading to fewer iterations on these cells.

However, implementing this approach introduces a whole new problem. Since cells have different time step sizes, they will also end up referring to different points in time in the simulation (i.e., different timestamps). The Riemann solver, however, needs the quantities of the cells to refer to the same timestamp in order to calculate the flux correctly. Otherwise, the simulation would calculate with quantities that don't match, violating conservation laws. With global time stepping, this was never a problem because the whole domain progressed simultaneously, so all cells always referenced

the same timestamp. In contrast, having different timestamps for each cell (using local time steps) means we need to receive the correct values from neighboring cells so that a given FV cell can proceed with the calculation in the first place. This is a classical interpolation problem.

In general, the FV solver needs the flux between the cells to perform a time step. The Riemann solver needs the information from the cell's neighbors at the timestamp of the cell to compute this flux. We already know that a cell can only proceed with computation if its neighboring cells are at least on the same timestamp; otherwise, we would use values that don't yet exist. Consequently, every time a cell can proceed with computation, its timestamp must be between the current and the last timestamp of all of its neighbors. This is because those neighbors, on the other hand, can proceed with their computation only when our original cell reaches their timestamp. Thus, we can calculate the values for the Riemann solver at the timestamp of the cell by considering only the current and previous time step of each neighbor. Using simple linear interpolation, we can retrieve the following equation (see Figure 2.5 for illustration):

$$q_k = q_{n-1} + (q_n - q_{n-1}) \cdot \frac{(t_k - t_{n-1})}{(t_n - t_{n-1})} \tag{2.5}$$

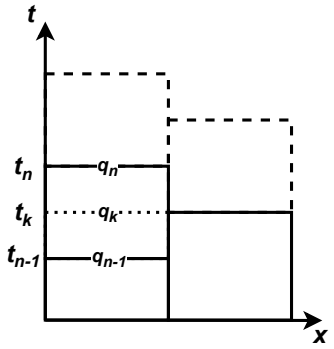


Figure 2.5: The cell on the right needs the value q_k of its neighbor at $t = t_k$ with $t_{n-1} < t_k < t_n$ to proceed with the calculation. Since the neighbor only calculated q_{n-1} and q_n, the cell needs to obtain this value via linear interpolation.

This problem of having to wait for each neighbor before a cell can proceed with computation can heavily impact the performance of our computation, depending on how well the cells are synchronized. On the one hand, synchronized cells can often compute simultaneously (see Figure 2.6), whereas unsynchronized cells constantly need to wait for each other (see Figure 2.7), leading to more iterations and therefore higher computational costs. This gets even more of a problem the more dimensions

(and therefore neighbors) you have. Therefore, having a strategy for synchronizing cells is crucial for an efficient local time stepping implementation.

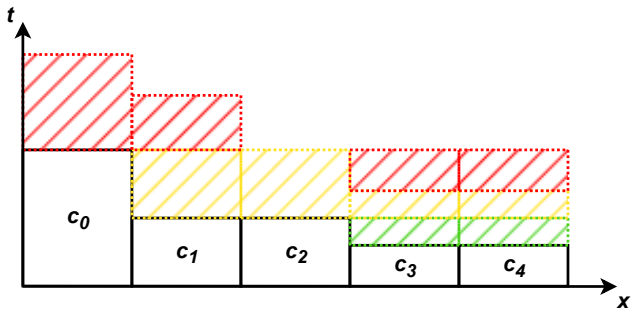


Figure 2.6: Example of a rather synchronized domain in local time stepping. Future time steps are shown by order (green, then yellow, then red). It needs fewer iterations due to the synchronized cells.

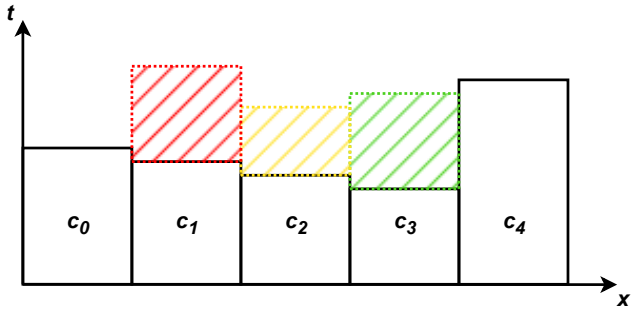


Figure 2.7: Example of a domain with unsynchronized cells in local time stepping. Future time steps are shown by order (green, then yellow, then red). The local time steps vary only slightly, so most cells only wait for their neighbors. This "waiting line" cascades through the domain (see cell c_3 down to c_0).

3 Related Work

In the last chapter, we saw that synchronization between neighboring cells is essential for constructing a local time stepping strategy. As can be seen in this chapter, research has already presented promising strategies and results. On the other hand, it is crucial to design an implementation that accounts for the limitations of the desired application. For us, it is essential to evaluate the applicability of a given approach in ExaHyPE. In this chapter, we explore different local time stepping strategies in research and determine whether they are feasible in the ExaHyPE architecture. Unfortunately, most of the related work focuses on solvers other than the finite volume method; however, it is possible to adapt their ideas to fit our needs. Additionally, we want to see how well other strategies perform in general compared to global time stepping to gain a sense for the possible increase in performance that a well-fitting local time stepping implementation might achieve.

3.1 Synchronization of Cells

The biggest advantage of local time stepping comes from saving computation time due to larger time steps in most of the cells. This advantage, however, is greatly reduced if the cells' time steps only vary slightly (see Figure 2.7), and they have to wait for each other because of that.

The work of Faille et al. [9] presents a local time stepping strategy that tries to minimize slight variations in time step sizes. Their method uses domain decomposition for the finite volume scheme, dividing the domain into a coarse and a refined subdomain. The coarse subdomain uses a large time step δt_2, while the fine subdomain uses a smaller time step δt_1 with $\delta t_2 = \mathcal{K} \cdot \delta t_1$. The integer $\mathcal{K}$ achieves temporal scaling for the two domains. This way, the refined subdomain is synchronized with the coarse subdomain exactly after $\mathcal{K}$ iterations. While it minimizes the need for synchronization effectively, the subdomains and time step sizes must be predefined before the simulation starts. It makes their approach a combination of local time stepping and global fixed time stepping, but it is also very unsuitable for a solver in ExaHyPE, because ExaHyPE is designed as a high-level engine that abstracts underlying complexities, such that the user shouldn't need to manually define more than the PDE system itself. Therefore, we didn't look further into this approach.

Ensuring synchronization, however, by grouping cells into clusters, is a common strategy in other research as well. All cells in a single cluster share the same time step size, so it is only necessary to synchronize across different clusters. Depending on the cluster size, this can significantly reduce the waiting overhead while also leveraging the advantages of individual local time steps. Both Breuer and Heinecke [10] as well as Krenz [11] implement clustered local time stepping based on the ideas presented by Breuer et al. [12].

The basic idea is to define a set of $N_C \geq 1$ different time clusters. While [12] generates as many clusters as needed to collect all time step sizes, [10] states that, in most cases, no more than three to five clusters are needed and allows the user to specify the number of clusters. The time step size of these clusters depends on the minimum time step in the domain t_{min}. We can define the time step size of cluster k as $r^k \cdot t_{\text{min}}$. The integer $r \geq 2$ defines the rate at which clusters increase their time step size. Mostly, the rate is set to $R = 2$. By ensuring that the next-largest cluster's time step size is a multiple of its predecessor's, the cells of neighboring clusters end up synchronized. Compared to domain decomposition [9], this approach doesn't require predefined subdomains or time step sizes, making it a highly dynamic strategy that is suitable for ExaHyPE.

To reduce the number of possible synchronization scenarios, [12] normalizes the clusters in the domain. This means they allow each neighboring cell to differ by no more than one cluster. For example, if cells c_1 and c_2 are neighbors and c_2 is in the cluster with time step $2^k \cdot t_{\text{min}}$, then c_1 is only allowed to be in clusters having time step $2^{k-1} \cdot t_{\text{min}}$, $2^k \cdot t_{\text{min}}$, or $2^{k+1} \cdot t_{\text{min}}$. [11] also enforces this rule and refers to it as the maximum difference invariance. In [10], the overhead of enforcing this invariance was no more than 1.5% while circumventing corner cases in its implementation. This effect is probably even more noticeable in higher-dimensional domains where cells have more neighbors.

Unfortunately, the maximum difference invariance is difficult to enforce in ExaHyPE. The grid traversal allows communication only through faces, so cascading the invariance might require multiple iterations in the worst case, resulting in significant overhead. Because of this, we didn't look further into implementing this invariance.

3.2 Dynamic Scheduling

Given that cells waiting for each other is the main problem, scheduling cell updates rather than iterating over the domain to eliminate this problem entirely seems apparent. Since all the timestamps and the next time step sizes are known, it is possible to dynamically construct a schedule for cluster updates such that no cell has to wait for any other cell.

Both [12] and [11] included scheduling in their implementations, however, in different forms. While [12] assigns operations to work items that are dynamically generated and reordered, [11] constructs an actor model that is essentially a state machine to determine the behavior of clusters in their respective state. Generally, both approaches replace the "normal" grid traversal of the domain to efficiently schedule the cell clusters. However, if these clusters are relatively small (or there are many clusters), large jumps in the domain could lead to many cache misses in shared memory. Consequently, memory latency might negate the efficiency gains from reducing waiting overhead, as stated in [11]. In general, this approach seems promising, but it requires careful consideration and planning when implemented.

Unfortunately, custom scheduling of cell updates is not really suitable for ExaHyPE. The entire engine relies on strict grid traversal in Peano curves, and although action sets have guards (if statements) that decide whether code for the cell event is executed or not, the grid traversal itself is very rigid [13]. This means that we can only decide if we want to execute code during a traversal, but we cannot change the traversal itself. Breaking up this foundation of the engine to implement custom scheduling would theoretically work, but it means we would have to rewrite much of the Peano engine itself. Since this is not the objective of this paper, we didn't look further into this approach.

3.3 Potential Speedup

When it comes to performance increase, most papers compare their local time stepping strategy to an implementation of global time stepping. The most interesting metric to us is the practical speedup in computation time, rather than a theoretical speedup based on assumptions and models. However, this metric depends heavily on the scenario tested and the underlying framework, so it should be taken with a grain of salt. In the research presented, practical speedups range from $3.2\times$ in [12] to $10.37\times$ in [10]. In general, these results suggest that reasonable improvements in ExaHyPE are possible by modifying a global to a local time stepping strategy. However, we cannot directly apply improvements from other research to our environment and therefore need to develop our own implementation and evaluate it.

4 Implementation

When implementing a strategy for local time stepping in ExaHyPE, we need to consider the engine's underlying infrastructure. In this chapter, we present our modifications to the existing global adaptive time stepping FV solver in order to achieve local time stepping in ExaHyPE. For simplicity, we will only focus on the relevant details; for example, this does not include mesh refinement or boundary conditions. The entire structure of ExaHyPE and Peano is documented in [1], [2], and [13]. The strategies for synchronization itself are presented in section 4.4. In total, we implemented three different approaches that are partially extensions of the strategies discussed in chapter 3. But first, the focus lies on achieving local time steps in the first place.

ExaHyPE's FV solver uses Peano cells as patches of $p \times p$ volume cells [13] (see Figure 4.1). Analogous to the documentation in [13], we will use the terms "patch" and "cell" interchangeably and "volume" for the inner FV cells within a patch. The data of the cells (e.g., timestamp, position) is only saved for the entire patch, not for the volumes within; they only store the patch quantities Q (i.e., the vector of conserved variables and parameters). Hence, these patches roughly act as clusters (not to be confused with time step clusters) of FV volumes that always have the same timestamp as well as the same time step size. The faces in between the cells store shared information about each cell on its side (i.e., between neighboring cells).

ExaHyPE builds on top of the underlying Peano infrastructure and grid traversal to update the patches. At the beginning of the simulation at timestamp $t = 0$, every cell starts with its initial condition (i.e., its base values). When the solver decides to perform a time step, it traverses the grid of patches. While traversing the grid, action sets can subscribe to certain events. An example is the `TouchCellFirstTime` event, which is invoked on every cell the first time it is visited in the traversal (see more in [13]). Action sets subscribed to these events are executed when the event is invoked. In coordination, these action sets execute the code that enables the patches to take time steps. An overview of this process is shown in Figure 4.2.

First, the action set `ProjectPatchOntoFaces` iterates over a cell's faces and copies the current patch quantities into the corresponding buffer. It also copies data like the current timestamp of the whole cell into the buffer. Afterwards, `RollOverUpdatedFaces` organizes the copied cell values from the buffer into the corresponding fields. In the next grid traversal, every updated cell value that needs to be exchanged is stored in the

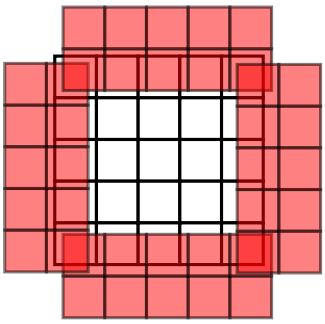


Figure 4.1: A patch of $p \times p$ volume cells with (here, $p = 5$). The overlying red volumes represent the constructed halo around the patch. Each side of the halo consists of a $2 \times p$ long strip of volume cells. This illustration is inspired by the Peano documentation of the finite volume scheme [13].

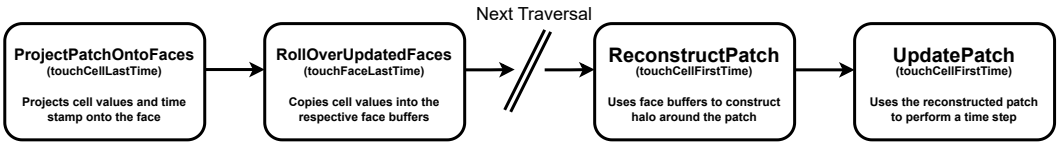


Figure 4.2: Process of a patch performing a time step, including the relevant action sets. `ProjectPatchOntoFaces` theoretically comes directly after `UpdatePatch`. However, since it is the first action set in the process of performing a new time step, we listed it first.

right faces. `ReconstructPatch` uses these values in the faces around the cell to create a halo (see Figure 4.1) of volume cells around the patch of the cell. Patches need this layer of the neighboring cells because they can only access their own volume cells. Lastly, the cell uses this halo, along with its own patch quantities, to perform the actual time step in `UpdatePatch`.

For the global adaptive time stepping method, these are the essential steps to perform a time step on the entire domain. In the next sections, we will adapt this process to achieve local time stepping.

4.1 Guard for Patch Computation

Each action set is protected by a guard (an if statement) that determines whether the action set is executed. When using global adaptive time stepping, every cell is allowed to compute during a grid traversal. As seen in chapter 2, this is not always the case with local time stepping, so we need to extend the guard of the action sets to only compute when they are allowed to.

A cell can only proceed with computation if it has the corresponding values of its neighbors. This means we can allow a cell with timestamp t_k to update only if the minimum timestamp of its neighbors, t_n, satisfies $t_n \geq t_k$. Additionally, if the cell cannot perform a time step, we also do not need to construct the halo around the patch. Therefore, we can add a boolean field to the cell data, named `shouldUpdate`, to enforce this rule. The guards of the action sets `ReconstructPatch` and `UpdatePatch` are then extended to execute only when `shouldUpdate == true`.

For this to work, the field still needs to be set correctly. To do so, we insert a new action set, called `CheckShouldUpdate`, that subscribes to the `touchCellFirstTime` event and is executed before `ReconstructPatch` and `UpdatePatch` (see Figure 4.3). In this action set, we compare the cell's timestamp with those of its neighbors (stored in the respective faces) to set `shouldUpdate` accordingly. This way, cells are updated only when they have the corresponding values.

4.2 Interpolation between Neighboring Patches

As already stated in chapter 2, a patch's timestamp t_k will always have the following property. Let t_n be the current timestamp of a fixed neighboring cell and t_{n-1} its previous timestamp with $t_{n-1} \leq t_n$; then t_k will always be in the interval $[t_{n-1}, t_n]$. Consequently, we need both the previous timestamp and the previous patch quantities (at t_{n-1}) in addition to the current ones in order to calculate the interpolated value (see Figure 2.5).

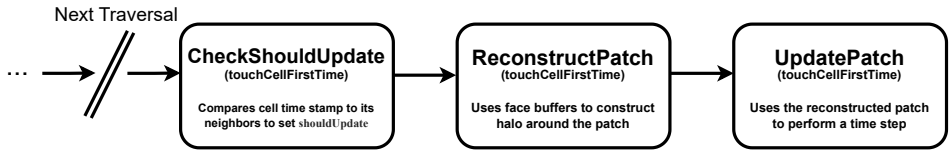


Figure 4.3: Extended process of performing a time step. This illustration shows the action set to set the value of the guard for `ReconstructPatch` and `UpdatePatch`. Given limited space, `ProjectPatchOntoFaces` and `RollOverUpdatedFaces` are deliberately hidden (see Figure 4.2).

To achieve that, we need to store these values in an additional buffer. However, we only need to store them on the faces, since interpolation is only required for the halo (not for the whole patch). Hence, the action set `ProjectPatchOntoFaces` doesn't need to be changed. On the other side, `RollOverUpdatedFaces` has to check if the face buffer has been updated. If so, it first moves the currently stored values to the field that contains the previous time step and patch quantities. Afterwards, it copies the buffer values of the time step performed in this traversal into the respective field. This way, the faces always hold both the current and the previous time step values.

Lastly, we need to use these values on the faces to compute the correct halo values. To do so, we modify `ReconstructPatch` to iterate over all faces of the cell. For each face, it first calculates the interpolation factor by using the stored timestamps and then interpolates the patch quantities to obtain the correct values (see Equation 2.5). As a result, the halo stores the interpolated values for the cell's timestamp.

4.3 Local Time Steps

After the previous sections, the cells correctly interpolate their neighbors' patch quantities and only take a time step if allowed. The only thing left is actually using a local time step. To achieve that, we add another field to the cell to store its current local time step size. The action set `UpdatePatch` then always uses this field instead of the globally saved admissible time step when performing a time step. By using the CFL rule [7] and Equation 2.4, we can directly calculate the new time step Δt_{CFL} for this cell according to the following equation:

$$\Delta t_{\text{CFL}} = \text{CFL}_N \cdot \frac{h_{\max}}{|\lambda_{\max}|} \tag{4.1}$$

Δt_{CFL} is then stored in the corresponding field and can be used in a future grid traversal when performing the next time step. If the maximum eigenvalue of the cell $\lambda_{\max}$ is 0, the equation is undefined. In this case, the cell simply uses the admissible

time step (i.e., the global adaptive time step).

4.4 Synchronization of Neighboring Patches

As stated previously, synchronizing neighboring cells to reduce unnecessary waiting overhead is crucial for improving computation time in local time stepping. In chapter 3, we already discussed some approaches to deal with this problem. Next, in the following subsections, we present three different local time stepping strategies that we implemented in ExaHyPE.

4.4.1 Time Step Tolerance

In Figure 2.7, it is prominent that an unnecessarily high amount of waiting compared to the time saved because of local time steps occurs when neighboring cells are only slightly out of sync. We can accommodate that if we introduce a certain tolerance ϵ with $\epsilon \geq 0$ for cells to synchronize with each other when performing a time step. To accomplish this, we want to consider the maximum timestamp of the neighboring cells, $t_{\max}$, and extend Equation 2.4 to the following:

$$\Delta t = \begin{cases} t_{\max} - t, & \text{if } \; t_{\max} \in [t + (1-\epsilon) \cdot \Delta t_{\text{CFL}},\; t + (1+\epsilon) \cdot \Delta t_{\text{CFL}}] \\ \Delta t_{\text{CFL}}, & \text{otherwise} \end{cases} \tag{4.2}$$

t is the current timestamp of the cell, and Δt_{CFL} is its calculated local time step (see Equation 4.1). This way, each cell might synchronize itself with the neighbor that is the furthest in time if its timestamp is inside the tolerance interval in order to reduce both their waiting times (see Figure 4.4). However, because the upper bound of this tolerance interval is greater than our intended local time step, this could lead to an unstable time step size. Consequently, we need to ensure that Equation 2.4 still holds by choosing ϵ according to the following:

$$(1+\epsilon) \cdot \text{CFL}_N < 1 \tag{4.3}$$

We decided to synchronize with the neighbor that has the largest timestamp, because this way, we are synchronized with the furthest neighbor, and for every other neighboring timestamp t_k and our timestamp t, $t_k \leq t$. Thus, no neighbor has to wait for this cell in the next traversal.

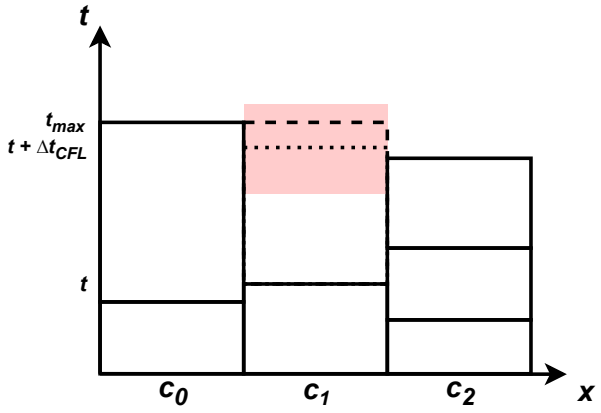


Figure 4.4: Example of the time step tolerance implementation. Cell c_1 has timestamp t. It would normally proceed with a time step size of Δt_{CFL} (to the dotted line), but because the furthest neighbor timestamp $t_{\max}$ of c_0 is within the tolerance (red-shaded area), c_1 adjusts its time step size to reach $t_{\max}$ (dashed line). This way, c_0 doesn't have to wait for c_1 in the next traversal.

4.4.2 Clustering

Clustering is a very prominent approach in research (see chapter 3). The strategy tries to collect cells with similar time step sizes into groups (clusters) that share the same time step. Assuming they start at the same timestamp, every cell in a cluster would be synchronized. Additionally, you would only have to deal with a limited number of different time step sizes. Whereas, if you had only one cluster, the entire domain would use the smallest possible time step (one shared time step), and the simulation would behave just like global adaptive time stepping.

In general, our clustering implementation is based on the schemes described in [12], [11], and [14]. Let's say we have m clusters in total. We then introduce $\Delta t_{\min}$ as the time step of the minimum cluster (the minimum cluster time step). The integer r represents the rate at which the cluster size is growing. Next, we can define the clusters as time step intervals:

$$
\begin{aligned}
Cl_0 &= [\Delta t_{\min},\ r \cdot \Delta t_{\min}) \\
Cl_1 &= [r \cdot \Delta t_{\min},\ r^2 \cdot \Delta t_{\min}) \\
&\dots \\
Cl_{m-1} &= [r^{m-1} \cdot \Delta t_{\min},\ \infty)
\end{aligned}
$$

When calculating their local time step, cells determine which cluster they belong to and always use the lower bound of that cluster's time step interval. For example, consider a rate of $r = 2$. A cell with a local time step of $\Delta t = 5 \cdot \Delta t_{\text{min}}$ would fall into cluster $Cl_2 = [2^2 \cdot \Delta t_{\text{min}}, 2^3 \cdot \Delta t_{\text{min}})$ and would therefore use a time step of $\Delta t = 2^2 \cdot \Delta t_{\text{min}}$.

Consider Δt_{CFL} as the minimum local time step across the entire domain (i.e., the admissible time step in global adaptive time stepping). When starting the simulation, we set $\Delta t_{\text{min}} = \Delta t_{\text{CFL}}$. Because Δt_{CFL} can change throughout the simulation, we need to dynamically adjust Δt_{min}. At the end of every grid traversal, when Δt_{CFL} has been calculated, we update the minimum cluster time step according to the pseudocode in Algorithm 1.

Algorithm 1 Pseudocode for dynamically adjusting the minimum cluster time step. dt_{CFL} is the admissible time step, dt_{min} is the minimum cluster time step, and r is the cluster rate.

```
while dt_CFL ≥ dt_min · r do
    dt_min ← dt_min · r
end while
while dt_CFL < dt_min do
    dt_min ← dt_min / r
end while
```

After updating the minimum cluster time step, $\Delta t_{\text{CFL}} \in Cl_0$ always holds. Additionally, all cluster time steps will remain a multiple of their previous cluster (according to the rate r).

Because a cell's local time step is also dynamic and changes throughout the simulation, cells can switch clusters. Considering synchronization, switching to a lower cluster (i.e., decreasing the time step) is not much of a problem (see Figure 4.5). On the other hand, switching to a higher cluster (i.e., increasing the time step) can cause the cell to have an "offset" if it is not properly aligned with its cluster, leading it to be unsynchronized with the domain (see Figure 4.6). We can resolve this issue by having the cluster's stamp be a multiple of its time step size. Then, by adjusting the cell's time step to follow this rule (see Algorithm 2), the cell automatically synchronizes with its cluster. And because the time step of each cluster is a multiple of that of the previous clusters, the entire domain is synchronized.

4.4.3 Consecutive Patch Updates

Sometimes, cells could continue computation, even if they had already performed one time step. This happens when a cell's timestamp after performing a time step is still smaller than the minimal timestamp of its neighbors. It still has all the information

Algorithm 2 Pseudocode for synchronizing the local time step of a cell with the cluster after switching to a higher cluster. dt is the cell's local time step, t is the cell's timestamp, and dt_{cl} is the time step of the cell's cluster. mod refers to the modulo operator.

```
1: // determine cluster for cell
2: ...
3: // synchronize time step size with cluster
4: dt ← dt_cl − (t mod dt_cl)
```

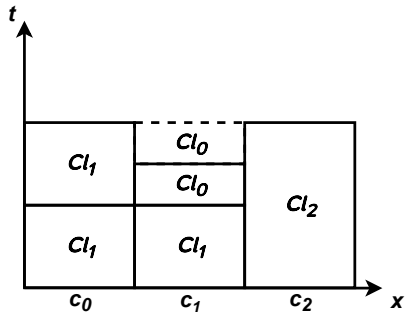


Figure 4.5: Example of a cell switching to a lower cluster. Cell c_1 switches from cluster Cl_1 to cluster Cl_0 and is still synchronized with the domain.

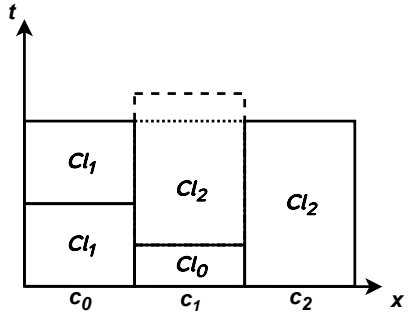


Figure 4.6: Example of a cell switching to a higher cluster. Cell c_1 switches from cluster Cl_0 to Cl_2, but because it is not aligned with cluster Cl_2, it needs to adjust its time step. The normal cluster time step (dashed line) is reduced to synchronize with the domain (dotted line).

and values it needs to further proceed in time (see Figure 4.7), but unfortunately, our current process scheme (Figure 4.2 or rather Figure 4.3) only allows for one time step per grid traversal. This is because `ReconstructPatch` only constructs the halo once for the current timestamp of the cell. Consequently, `UpdatePatch` needs to wait for the next grid traversal for `ReconstructPatch` to interpolate the neighboring values at its next timestamp.

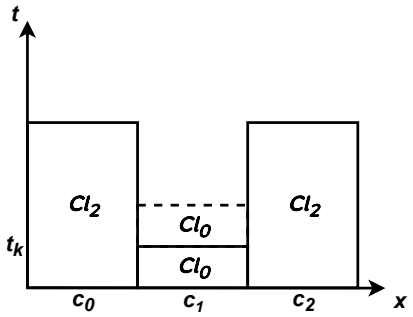


Figure 4.7: Example situation in which the cell c_1 currently has the timestamp t_k immediately after performing a time step and would remain in cluster Cl_0. It would have all the information it needs to perform another time step (to the dotted line), but the current process only allows for one time step per grid traversal.

However, we can adjust our process scheme to allow consecutive patch updates within one grid traversal by using our clustering implementation and expanding it. We combine `ReconstructPatch` and `UpdatePatch` into `ReconstructUpdatePatch` (see Figure 4.8) so that each cell can keep computing as long as it has the necessary information. This means it can perform time steps until it needs to wait for at least one neighbor.

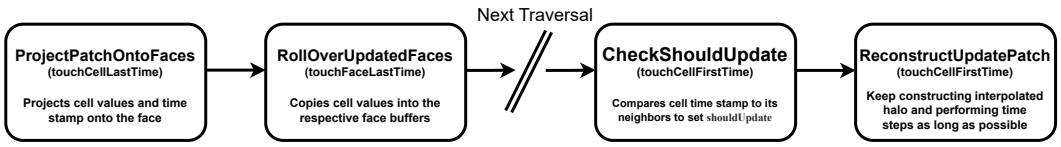


Figure 4.8: Process scheme for consecutive patch updates. It takes the process scheme in Figure 4.3 and merges `ReconstructPatch` and `UpdatePatch` into `ReconstructUpdatePatch` to perform multiple time steps within one grid traversal.

The execution scheme of `ReconstructUpdatePatch` itself is displayed in Algorithm 3.

Each cell follows a loop that performs the reconstruction (interpolation of neighboring patch quantities) and time step of the patch until $t > t_n$, with t as the cell's timestamp and t_n as the smallest neighbor timestamp. The result is lower overhead due to cells waiting for neighboring cells that could continue computation after one time step (see Figure 4.9).

Algorithm 3 Pseudocode for the action set `ReconstructUpdatePatch`. t is the timestamp of the cell, dt is the local time step the cell is using, and t_n is the smallest neighbor timestamp. The cell proceeds performing time steps until it passes t_n.

```
1:  while t ≤ t_n do
2:      /* Get timestamps from neighboring cells to
3:      construct interpolated halo of volumes around
4:      the patch. */
5:      ...
6:      // calculate time step size dt
7:      ...
8:      // update patch with dt and the constructed halo
9:      ...
10:     t ← t + dt
11: end while
12: // save updated values to cell data
13: ...
```

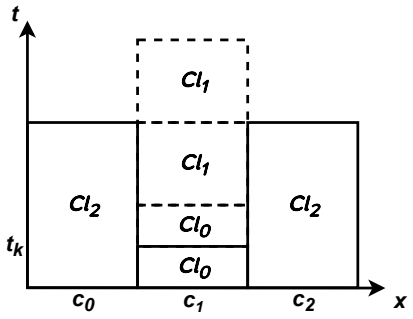


Figure 4.9: Adapted example situation of Figure 4.7. Cell c_1 continues computation after its first time step at t_k. Right after its second time step in this traversal, it switches to cluster Cl_1 and can perform another two time steps.

5 Evaluation

In the last chapter, we described three different implementations of local time stepping in ExaHyPE. For clarification, we name them *LTStolerance*, *LTSclustering*, and *LTSconsecutive* respectively, in the order we introduced them. Likewise, we abbreviate the existing implementation of global adaptive time stepping as *GTSadaptive* and global fixed time stepping as *GTSfixed*. Now, to evaluate our implementations against the existing FV solvers in ExaHyPE (*GTSadaptive*, *GTSfixed*), we run simulations of four different scenarios.

Each implementation is tested with multiple configurations. For instance, using *LTStolerance*, we can change the tolerance factor ϵ (see Equation 4.2), whereas for *LTSclustering* and *LTSconsecutive*, we can change the number of clusters m as well as the rate r (see subsection 4.4.2). We test different configurations of our implementations to determine which works best in each scenario. For *GTSfixed*, we always use the smallest time step from *GTSadaptive* as the fixed time step to ensure comparability.

The metric we are interested in is the time the solver takes to compute the simulation. We run our simulations on the *CoolMUC-4* Linux cluster [15], located at the Leibniz Supercomputing Centre in Munich, Germany. The cluster segment we used (*cm4*) runs on Intel(R) Xeon(R) Platinum 8480+ (Sapphire Rapids) nodes with 112 physical cores and 512 GB of memory per node. All of our simulations were computed on one node, using 81 physical cores for 1 MPI rank and 81 OpenMP threads.

Since time step sizes depend heavily on the cells' eigenvalues, we chose scenarios that reflect two different kinds of eigenvalue changes: spatial and temporal. Spatial change means that at a fixed point in time, different cells hold different eigenvalues, whereas temporal change means that for a fixed cell, its maximum eigenvalue changes throughout the simulation (i.e., over time).

In the following sections, we will present each scenario. However, when defining the PDE systems, we will focus primarily on the behavior of the maximum eigenvalues, as they are most important for our considerations. The interested reader is referred to the work of Leveque [5].

5.1 Scenario 1: 3D Elastic Shock

For the first scenario, we look at linear elastic wave equations [5]. These equations describe the propagation of elastic waves in a solid medium governed by particle velocity v and material stress σ. For simplicity, we only define the equations for the 2-dimensional case. The conserved quantities Q are written in the form

$$Q = \begin{pmatrix} \sigma_{xx} & \sigma_{yy} & \sigma_{xy} & v_x & v_y \end{pmatrix}^\top. \tag{5.1}$$

In addition, the PDE is defined as

$$\frac{\partial Q}{\partial t} + \nabla \cdot F(Q, \nabla Q) = \frac{\partial Q}{\partial t} + A\frac{\partial Q}{\partial x} + B\frac{\partial Q}{\partial y} = 0 \tag{5.2}$$

with the following matrices:

$$A = \begin{pmatrix} 0 & 0 & 0 & -(\lambda+2\mu) & 0 \\ 0 & 0 & 0 & -\lambda & 0 \\ 0 & 0 & 0 & 0 & -\mu \\ -\frac{1}{\rho} & 0 & 0 & 0 & 0 \\ 0 & 0 & -\frac{1}{\rho} & 0 & 0 \end{pmatrix} \quad B = \begin{pmatrix} 0 & 0 & 0 & 0 & -\lambda \\ 0 & 0 & 0 & 0 & -(\lambda+2\mu) \\ 0 & 0 & 0 & -\mu & 0 \\ 0 & 0 & -\frac{1}{\rho} & 0 & 0 \\ 0 & -\frac{1}{\rho} & 0 & 0 & 0 \end{pmatrix} \tag{5.3}$$

While ρ is the mass density, λ and μ are called Lamé parameters and control the material's properties. Specifically, they control the elastic wave speeds, namely those of the pressure waves $c_p = \sqrt{\frac{\lambda+2\mu}{\rho}}$ and shear waves $c_s = \sqrt{\frac{\mu}{\rho}}$. Given that the Lamé parameters are non-negative, $c_p \geq c_s$ always holds. Hence, the maximum eigenvalue $\lambda_{\max}$ will always be the pressure wave.

The first scenario uses these elastic equations to simulate two impulses in opposite corners in a 3-dimensional domain $\Omega = [0,1]^3$ (see Figure 5.1). The domain itself has tree depth (of Peano trees [13]) of 4. This results in $3^4 = 81$ patches along each axis, totaling $81^3 = 531,441$ patches in the domain. Each patch consists of $16^3 = 4096$ volume cells, yielding a patch size of $p = 16$.

For the initial condition ($t = 0$), we define the position of two Gaussian pulses at $x_A = (0,0,0)$ and $x_B = (1,1,1)$. The strength P that the pulse located at x_c has on a cell at position x is defined as

$$P(x, x_c) = \exp\left(-\frac{\|x - x_c\|^2}{w}\right) \tag{5.4}$$

where $\|\cdot\|$ denotes the Euclidean norm and $w = 0.02$ represents the width parameter

of the Gaussian. The velocity v is set to $v = 0$. On the other hand, the stress tensor σ is set to $\sigma_{xx} = \sigma_{yy} = \sigma_{zz} = P(x, x_A) + P(x, x_B)$ and $\sigma_{xy} = \sigma_{yz} = \sigma_{xz} = 0$.

These impulses move through a perfectly uniform material, having constant values for the density ρ as well as the two Lamé parameters λ and μ for the entire domain. As seen before, the maximum eigenvalue depends entirely on those three parameters, so every cell has the same eigenvalue, which doesn't change throughout the simulation (i.e., neither spatial nor temporal variation). Consequently, all cells have the same time step size, as determined by the CFL rule (Equation 4.1).

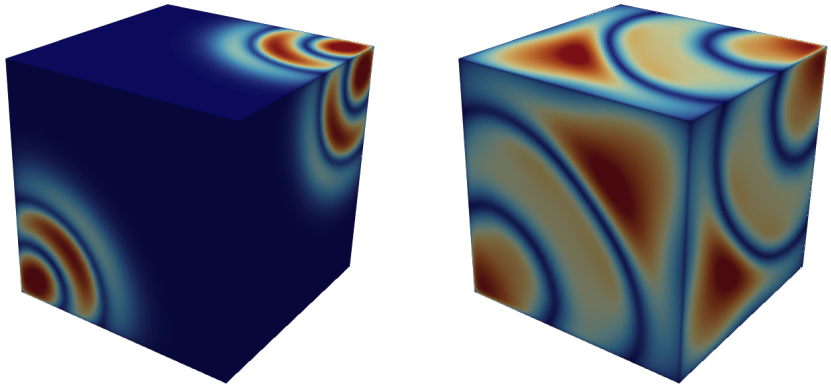


Figure 5.1: Visualization of the first scenario at different points in time. Two initial impulses start on opposite corners of the domain (left image) and propagate through a domain with homogeneous material properties until they collide (right image).

The results of scenario 1 are shown in Figure 5.2. It is prominent that all local time stepping implementations perform more than 2 times slower than the global time stepping variants. This mainly comes from the overall overhead of local time stepping, because there is no time saving from the time steps without any variance (spatial and temporal) in the eigenvalues. The overhead is probably even higher because we simulate a three-dimensional domain. In higher dimensions, more data needs to be transferred, interpolated, and checked between the cells. Furthermore, since cells have more neighbors in higher dimensions, the likelihood of them having to wait for one such neighbor generally increases.

GTSfixed is around 8% faster than *GTSadaptive* and therefore the fastest solver for scenario 1. This probably comes from the slight overhead that *GTSadaptive* has by calculating the eigenvalues as well as the admissible time step in each grid traversal. Because

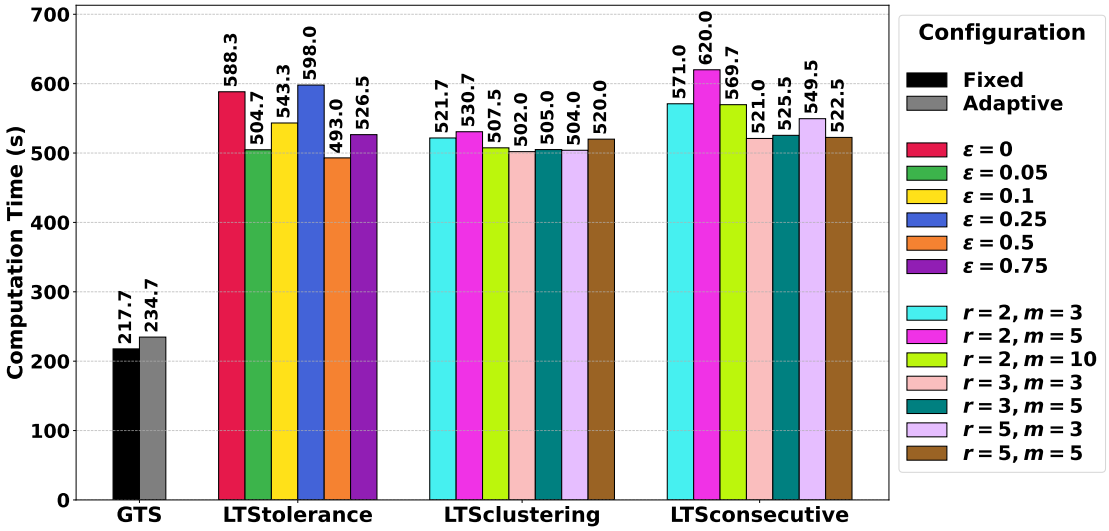


Figure 5.2: Results of scenario 1, 3D elastic shock.

the time steps don't change over time, there is no need to calculate the admissible time step, and *GTSfixed* performs the best. Moreover, within *LTStolerance* and *LTSconsecutive*, the computation time varies slightly more for different configurations compared to *LTSclustering*, but this probably just comes from some edge cases, depending on the grid traversal order.

In addition to the properties of spatial and temporal constant eigenvalues, this scenario demonstrates that our implementation of local time stepping is capable of computing in higher dimensional space ($d > 2$). A visual comparison between the numerical results of computing with *GTSadaptive* and *LTSclustering* is shown in Figure 5.3.

5.2 Scenario 2: Uniform Heating

The Euler equations [5] describe the dynamics of an inviscid, compressible fluid. This results in a non-linear system of hyperbolic conservation laws that describes the conservation of mass density ρ, momentum ρv, and the energy ρE in a fluid continuum. These quantities are included in the conserved quantities vector

$$Q = \begin{pmatrix} \rho & \rho v_x & \rho v_y & \rho E \end{pmatrix}^\top. \tag{5.5}$$

The system can then be expressed in the following PDE [16]:

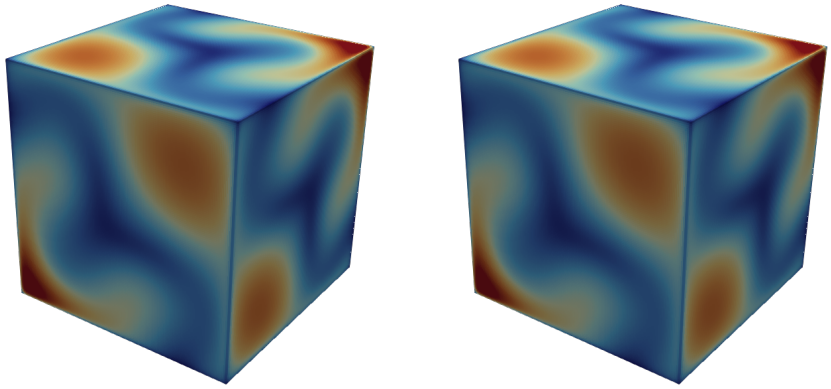


Figure 5.3: Comparison between computing scenario 1 using *GTSadaptive* (left) or *LTSclustering* (right) at the same timestamp. This indicates that the local time stepping implementation is suitable for higher dimensions (at least for $d = 3$).

$$\frac{\partial Q}{\partial t} + \cdot F(Q, \nabla Q) = \frac{\partial Q}{\partial t} + \frac{\partial G(Q)}{\partial x} + \frac{\partial H(Q)}{\partial y} = 0 \tag{5.6}$$

The PDE is defined by the following flux vectors:

$$G(Q) = \begin{pmatrix} \rho v_x & \rho v_x^2 + p & \rho v_x v_y & v_x(\rho E + p) \end{pmatrix}^\top \tag{5.7}$$

$$H(Q) = \begin{pmatrix} \rho v_y & \rho v_x v_y & \rho v_y^2 + p & v_y(\rho E + p) \end{pmatrix}^\top \tag{5.8}$$

Considering an ideal gas, the pressure p is determined by the internal energy of the fluid:

$$p = (\gamma - 1)(\rho E - \frac{1}{2}\rho v^2) \tag{5.9}$$

The adiabatic index (ratio of specific heats) γ is a medium constant (e.g., 1.4 for air). The prominent wave speed is determined by the acoustic speeds $\lambda = v \pm \sqrt{\frac{\gamma p}{\rho}}$. Unlike the linear elastic equations, where wave speeds remain constant over time, in the Euler equations, $\lambda_{\max}$ depends on the flow state itself. High pressures (high energy) or high velocities will dynamically increase the eigenvalue.

For the second scenario, we want to achieve high temporal variance with little to no

spatial variance in the eigenvalues. For this, we use the Euler equations to simulate a medium that heats up uniformly (i.e., gains energy). The domain $\Omega = [0,1]^2$ consists of $243^2 = 59,049$ patches with each patch containing $16^2 = 256$ volume cells. The initial condition ($t = 0$) has the following conserved quantity vector (compare with Equation 5.5):

$$Q = \begin{pmatrix} 1 & 0 & 0 & 2.5 \end{pmatrix}^\top \tag{5.10}$$

Now, we include ρE in the source term such that the energy density in every cell grows exponentially (see Figure 5.4):

$$S = \begin{pmatrix} 0 & 0 & 0 & \alpha\rho E \end{pmatrix}^\top \tag{5.11}$$

Here, the coefficient α helps us to control the growth rate of the energy density. We set $\alpha = 2.142857$ such that the maximum eigenvalue grows from $\lambda_{\max} \approx 1.18$ at $t = 0$ to $\lambda_{\max} \approx 50.3$ at $t_{\text{end}} = 3.5$. Because the fluid is static ($v = 0$) and density is constant ($\rho = 1$), the wave speed depends entirely on pressure, which is proportional to the energy (see Equation 5.9). This way, every cell has the same eigenvalue at a fixed point in time (no spatial variance), while the eigenvalue itself grows in orders of magnitude throughout the simulation (high temporal variance). In our case, the maximum eigenvalue grows by a factor of around 42. Consequently, the maximum stable time step size (Equation 4.1) shrinks by a factor of around 42 over the course of the simulation.

The results of scenario 2 are shown in Figure 5.5. Overall, *GTSadaptive* demonstrates the best performance, achieving nearly a two-fold speedup compared to *GTSfixed*. This comes from *GTSfixed* having to use a very small time step (from the end) for the entire simulation, while *GTSadaptive* dynamically adapts its time step. Additionally, *LTStolerance* performs almost as well as *GTSadaptive*, only around 10% worse. This is because *LTStolerance* also dynamically adapts to the higher eigenvalues, and because there is no spatial variance, there is no synchronization issue, so this difference arises purely from the local time stepping overhead. In this context, the overhead consists of the lookup of the neighbors' timestamps and, more significantly, the additional memory communication of the previous patch overlap alongside the current one between Peano subtrees. Consequently, the tolerance factor ϵ has no impact on the performance of *LTStolerance*.

However, because the cluster with the smallest time step, Cl_0, in *LTSclustering* and *LTSconsecutive* can only increase its time step if every cell is in the next higher cluster, these implementations perform much worse. Instead of dynamically adjusting to the highest possible time step in the domain, they have to wait and stay in Cl_0. Because the switch in the higher cluster is based on the rate r, the configurations perform drastically

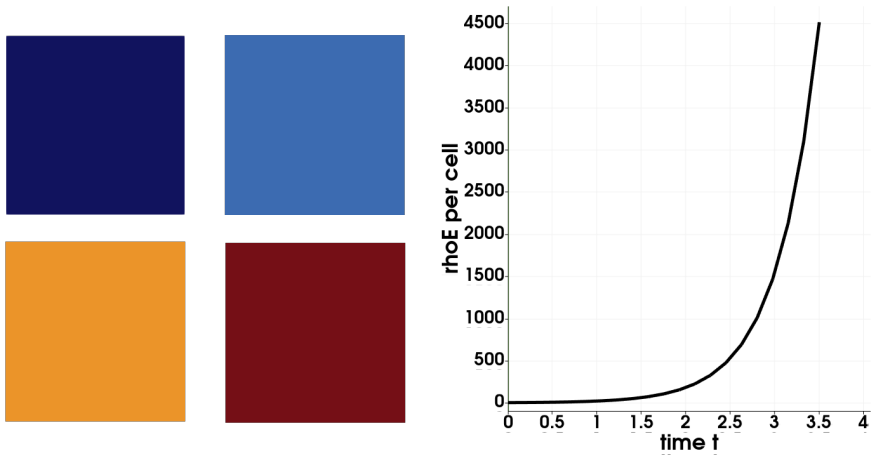


Figure 5.4: Visualization of the second scenario. On the left, you can see the domain at different points in time (chronologically from left to right, top to bottom). The plot on the right shows the energy density per cell, ρE, over time. You can see that the domain's energy grows exponentially, as does its eigenvalue, until the termination time at $t = 3.5$.

worse as the rate increases. Furthermore, because the entire domain is always in the lowest cluster, a larger number of clusters m has no real impact on computation time.

5.3 Scenario 3: Elastic Barrier

Just like the first scenario, the third scenario also uses linear elastic wave equations (see Equation 5.2). However, this time, we want to achieve high spatial variance with no temporal variance in the eigenvalue. The latter is achievable in a similar way to the first scenario by keeping the material properties constant over time, since the eigenvalue depends solely on them. Though we can choose these properties based on the cell's position to achieve high spatial variance.

In general, we simulate a 2-dimensional domain $\Omega = [0, 1]^2$ with an initial impulse on the left side at $x_A = (0, 0.5)$ that propagates through the material. The domain consists of $729^2 = 531,441$ patches, each consisting of $16^2 = 256$ volume cells. For the initial condition ($t = 0$), we set all conserved quantities to zero except for the horizontal stress σ_{xx}. Now, we calculate the strength P (see Equation 5.4) using $w = 0.005$ with

$$\sigma_{xx} = P(x, x_A). \tag{5.12}$$

However, there is a vertical strip of another material, 10% wide, in the middle of

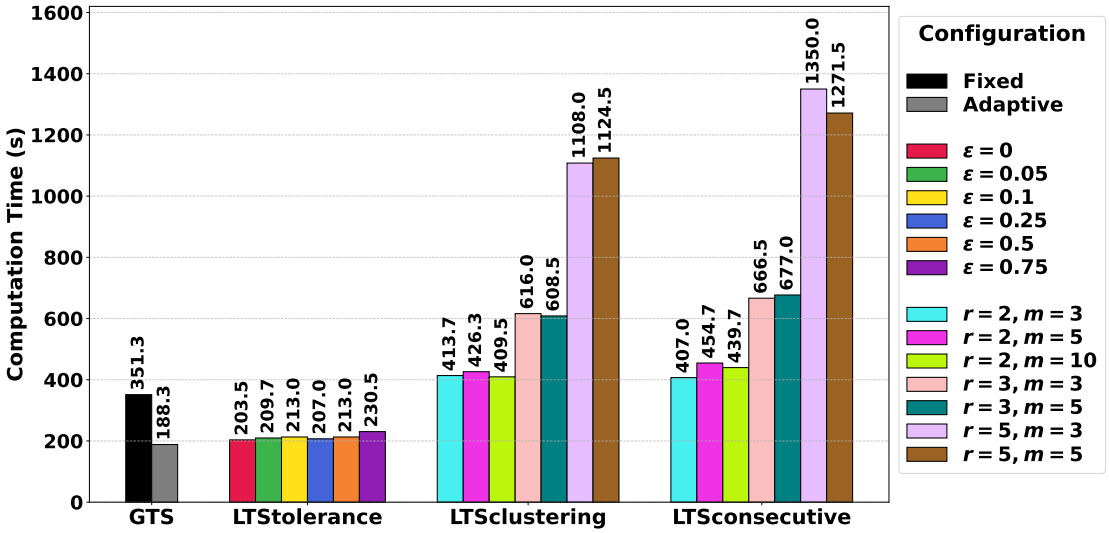


Figure 5.5: Results of scenario 2, uniform heating.

the domain with other material properties compared to the rest (see Figure 5.6). This barrier's eigenvalue is around 18 times higher than the rest of the domain, leading to a ($\approx 18\times$) smaller CFL time step (Equation 4.1) in this area.

The results of scenario 3 are shown in Figure 5.7. *GTSadaptive* performs best, with *GTSfixed* being only slightly slower. The local time stepping implementations are about 30% slower, with *LTStolerance* performing slightly faster than the other two. The difference to global time stepping probably comes from overhead and the fact that the time savings from local time steps are not large enough. *LTStolerance* is likely a bit more efficient than *LTSclustering* and *LTSconsecutive*, because the eigenvalues between the two materials vary by a factor of around 18, and *LTStolerance* makes use of the entire difference, while the cluster implementations reduce this factor to a power of the rate. Furthermore, the configuration with $r = 2$ and $m = 3$ only supports a factor difference of $2^{m-1} = 4$ between the lowest and highest clusters, which explains the large computation time compared to the other configurations. In general, the difference in eigenvalues probably isn't large enough to make local time stepping more efficient. Moreover, because the materials are rather dense (i.e., neighboring cells most likely have the same eigenvalue), there is no real improvement from *LTSconsecutive* compared to *LTSclustering*.

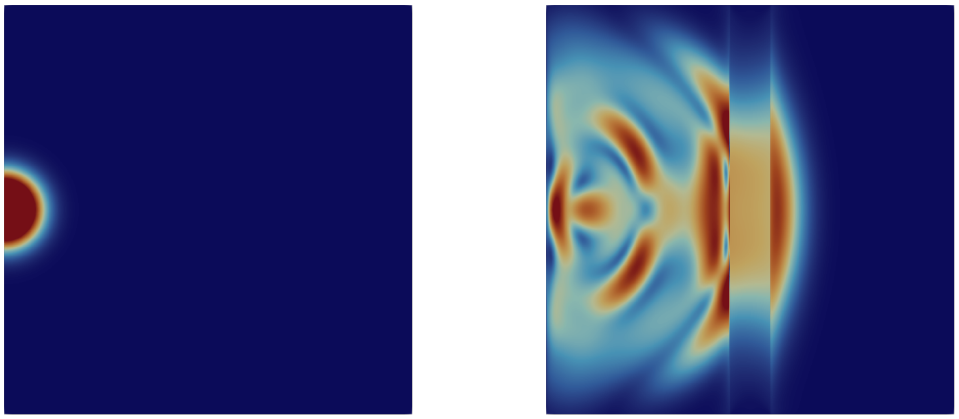


Figure 5.6: Visualization of the elastic barrier scenario. An initial impulse at $t = 0$ (left image) propagates through the domain, passing a 10%-wide barrier in the middle, which has a very high eigenvalue compared to the rest of the domain (right image).

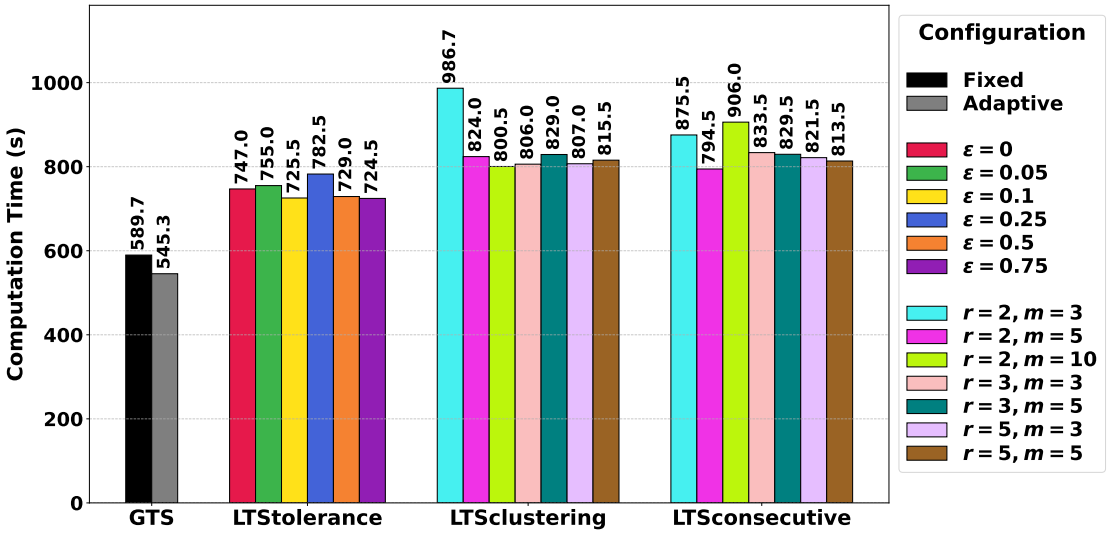


Figure 5.7: Results of scenario 3, elastic barrier.

5.4 Scenario 4: Three Mounds Channel

The shallow water equations [5] describe systems of fluids where the horizontal length scale is significantly larger than the vertical depth, making them standard for simulating tsunamis, tides, and dam breaks. The conserved quantities include the water column height (depth) h, the bathymetry b, and the water's depth-averaged velocity hv with the quantity vector

$$Q = \begin{pmatrix} h & hv_x & hv_y & b \end{pmatrix}^\top. \tag{5.13}$$

Notice that the bathymetry is constant (assuming the terrain doesn't change) and will not be affected by the flux. The wave propagation speed solely depends on these conserved quantities in addition to the gravity force g, resulting in $\lambda = v \pm \sqrt{g(h+b)}$.

Scenario 4 is primarily based on the "Three Mounds Channel" application example inside ExaHyPE. It uses the shallow water equations to simulate a 2-dimensional domain $\Omega = [0, 30]^2$ with a dam break on the left side and three mountains on the right side, arranged to form channels (see Figure 5.8). The domain consists of $729^2 = 531,441$ patches with each patch having $16^2 = 256$ volume cells. The initial water height h at $t = 0$ for a cell at position (x_x, x_y) is calculated as the following:

$$h(x, y) = \begin{cases} 0.9 & \text{if } x \leq 5.0 \\ 0.0 & \text{if } x > 5.0 \end{cases} \tag{5.14}$$

The initial water velocity hv is set to $hv_x = hv_y = 0$ for the whole domain. For the topology, we define three shape functions m_i based on the Euclidean distance of the cells' position x from specific centers c_i:

$$m_i(x) = 1.0 - k_i \|x - c_i\|_2 \tag{5.15}$$

The parameters for the three mountains are:

- Mountain 1: Center $c_1 = (15.0, 22.5)$, slope $k_1 = 0.10$.
- Mountain 2: Center $c_2 = (15.0, 7.50)$, slope $k_2 = 0.10$.
- Mountain 3: Center $c_3 = (28.0, 15.0)$, slope $k_3 = 0.28$.

To calculate the final bathymetry b that is used in the conserved quantities, we take the maximum envelope of these shapes, clamped at zero (sea level):

$$b(x) = \max(0, m_1(x), m_2(x), m_3(x)) \tag{5.16}$$

This configuration uses an augmented-state Riemann solver, originally proposed by David L. George [17], to handle the mathematical breakdown that occurs as water depth $h \to 0$. We use this scenario because of the rapid shifts in the eigenvalues. Overall, the system has a very high spatial variance due to the difference in channels and mounds, as well as a very high temporal variance, given the highly dynamic nature of the dam break.

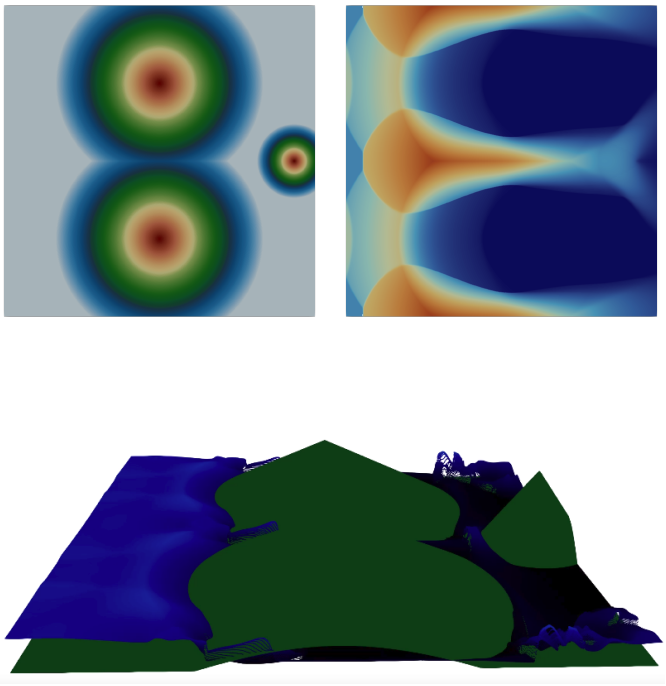


Figure 5.8: Visualization of the three mounds channel. The top-left image shows the positions of the three mounds. The color gradient visualizes the height difference. The top-right image shows water flowing through the channels between the mounds after the dam break. The bottom image shows a 3D visualization from the side. It is taken from the 3D visualization video in the ExaHyPE repository [1] and shows the simulation at a specific point in time. The bathymetry b is shown in green, and the water height h is shown in blue.

The results of scenario 4 are shown in Figure 5.9. Here, *GTSfixed* and *LTStolerance* with 0 tolerance (i.e., pure local time stepping) perform by far the worst. This scenario is highly dynamic, so pure local time stepping without any synchronization probably results in significant waiting overhead. On the other side, the other configurations of *LTStolerance* ($\epsilon > 0$) even outperform *GTSadaptive*, with $\epsilon = 0.5$ being about 40%

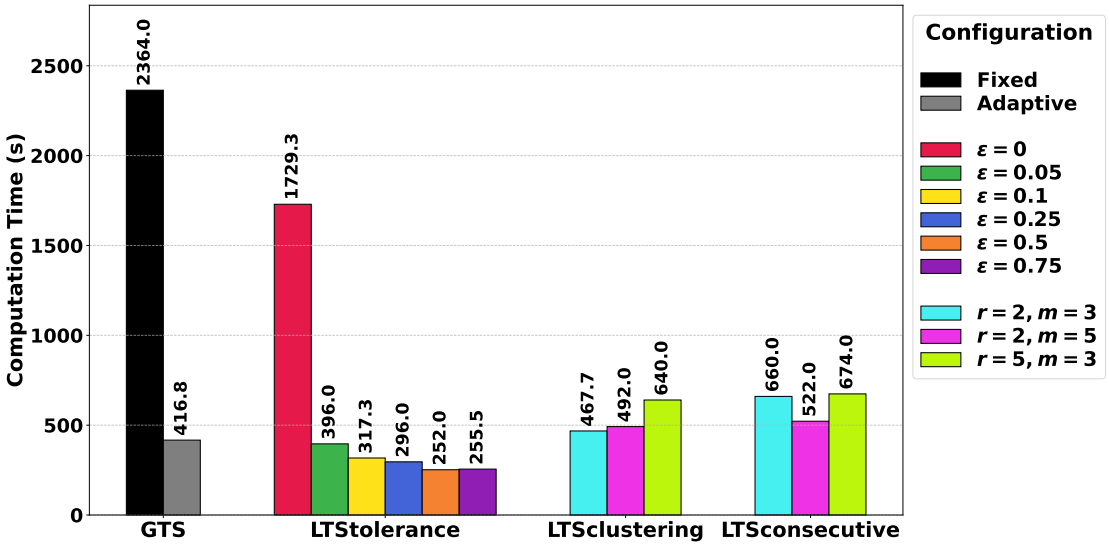


Figure 5.9: Results of scenario 4, three mounds channel. Note that some configurations of *LTSclustering* and *LTSconsecutive* are missing compared to the results in Figure 5.2, Figure 5.5, and Figure 5.7 because there were errors when trying to compute with these parameters. At some point in the simulation, all eigenvalues became zero, which indicates that the simulation became numerically unstable as the time steps grew too large. However, because the other results were sufficient and the missing configurations for *LTSclustering* and *LTSconsecutive* were unlikely to be too unusual, we didn't look further into resolving the issue.

faster. In general, the performance of *LTStolerance* improves with increasing tolerance factor ϵ until we set $\epsilon = 0.75$. It's highly likely that the advantage of synchronization with the cells' neighbors increases until $\epsilon = 0.5$. Afterwards, the tolerance window is so large that many cells reduce their intended time step size to accommodate the furthest neighbor. Then, the cost of reducing the intended time step size outperforms the performance gains of better synchronization.

Additionally, *LTSclustering* and *LTSconsecutive* are almost as good as *GTSadaptive* (about $10 - 30\%$ slower). However, the scenario probably isn't well-suited to clustering approaches, aside from the differences between the mounds and the water flow. The difference in the eigenvalues within the water flow is just not large enough and too sparse (i.e., large differences are randomly distributed in the domain) to form efficient clusters. This is also evident from the tiny difference in performance when changing the rate or the number of clusters.

5.5 Result Analysis

We already mostly discussed the results in the respective sections, so in this section, we want to summarize the findings to gain a broader view of the advantages and disadvantages of our local time stepping implementations.

In the first scenario, we simulated a three-dimensional domain. However, the problem of unsynchronized neighbors becomes worse as the dimension increases. This is mainly because each cell has more neighbors ($= 2 \cdot d$), leading to more possible combinations in which a cell has to wait for its neighbors. Furthermore, since there are more faces in higher dimensions, more data needs to be transferred between the trees in ExaHyPE. This issue becomes even worse when using local time stepping, because the faces store double the information (for the current and previous time steps), leading to higher memory demand. So in general, local time stepping works in higher dimensions, but the possible efficiency gain is probably harder to achieve than in lower dimensions.

Overall, *LTStolerance* performed the best of the three implemented local time stepping strategies. First, it was the only implementation that outperformed *GTSadaptive* in one of the scenarios. Second, in all the other scenarios (except for the first 3D scenario), the difference to *GTSadaptive* was not too significant. Generally, *LTStolerance* is a highly dynamic implementation of local time stepping that incurs only a modest overhead in unsuitable scenarios. This is because individual cells constantly try to take their best possible time step and synchronize based on their surrounding neighbors. Without custom scheduling or better control over the traversal in ExaHyPE, this dynamic nature of *LTStolerance* becomes a great strength.

Unfortunately, both clustering-based implementations (*LTStclustering* and *LTSconsecutive*) didn't perform as well as *LTStolerance*. Still, in the last two scenarios, both clustering approaches came close to the performance of *LTStolerance*. However, especially in the first two scenarios, it became clear that scenarios where the variation of the cells' eigenvalues is rather small are not suited for clustering approaches. Because changes in clusters depend on the rate (which is at least 2), changes in eigenvalues over time need to be drastic to benefit from the synchronization achieved by assigning time intervals (clusters) to cells. Moreover, there needs to be a high spatial variance with clear segmentation, so that clusters can form orderly. In general, the rather rigid traversal in ExaHyPE means that scenarios must be very well suited to clustering approaches for them to be more efficient than *LTStolerance*.

In addition to the poor performance of clustering in general, our proposed strategy of consecutive patch updates, *LTSconsecutive*, didn't achieve any improvement over the standard *LTSclustering*. This probably comes from the unlikelihood of the situation shown in Figure 4.7 to occur in $d > 1$ dimensions. It is just unlikely that the eigenvalue of one cell is much larger than all the surrounding cells. Usually, larger eigenvalues

affect groups of cells (clusters).

However, especially the fourth scenario demonstrated, that all of our synchronization strategies are more efficient than normal local time stepping using $\epsilon = 0$.

6 Conclusion and Outlook

This thesis explored the characteristics of the finite volume method using a local time stepping strategy in comparison to global time stepping approaches. We implemented the basic local time stepping scheme for the finite volume solver in ExaHyPE and introduced three strategies to address the synchronization issue between neighboring cells. For this, we explored proposed strategies in the research literature and even developed a unique approach (time step tolerance). Furthermore, we evaluated other strategies for their feasibility in ExaHyPE and made adjustments if necessary. Lastly, to evaluate the performance of our implementations against the existing FV solver in ExaHyPE and against each other, we conducted four scenarios and compared the results.

Our results indicated that the efficiency of our local time stepping implementations depends heavily on the scenario's eigenvalues and how they change. Generally, *LTStolerance* has proven itself to be the most adaptable of our implementations, achieving an efficiency improvement of about 40% compared to *GTSadaptive* in the fourth scenario. Unfortunately, most of the scenarios we tested weren't well-suited to clustering approaches, because the differences in the eigenvalues weren't large enough or the clusters were too sparsely distributed, so the synchronization cost more than the savings from larger time steps. However, the results indicated that our approaches for the synchronization issue achieved significant improvements over pure local time stepping without any synchronization strategy.

Still, there are many things that we didn't consider and could therefore be improved in future work.

For instance, we only looked at one version of time step tolerance. This version includes a tolerance in both directions (smaller and greater time step) for the maximum neighbor timestamp. However, other versions that include tolerance in only one direction (or different ϵ for each direction) could be interesting. Furthermore, if synchronization with the furthest neighbor didn't work, we could look at the second furthest neighbor (and so on).

When we looked at other implementations of time step clusters, the maximum difference invariance was prominent in most of them (see chapter 3). The premise of this invariance is to reduce possible synchronization issues. However, we didn't look further into enforcing it. Future work could try to implement and evaluate the effects

of this invariance in ExaHyPE.

When we implemented the local time stepping FV solver, we focused only on the computational aspects and didn't look into the plotting steps of the solver. When we then evaluated the local against the global time stepping results, it stood out that the plotting still doesn't look 100% correct, but instead lags behind or is generally not synchronized. This is probably because the solver plots a current "snapshot" of the domain's data at fixed time intervals. Consequently, further work needs to be done to improve the plotting for local time stepping (e.g., by using synchronization points).

Generally, this thesis's goal was to extend ExaHyPE by a new time stepping approach, which may be computationally beneficial in certain scenarios. We achieved this goal by implementing this time stepping strategy for the FV solver and by evaluating three approaches to the synchronization issue. However, further, more extensive testing could strengthen the understanding of fitting scenarios or even find flaws in our implementation that could lead to future improvements. On the other hand, other synchronization strategies we didn't look into further could also be explored. Lastly, based on this implementation for the FV solver, the next plausible step is to implement local time stepping for ExaHyPE's ADER-DG solver, as most of the research presented focuses on this solver compared to the FV method.

List of Figures

Bibliography

[1] A. Reinarz, D. E. Charrier, M. Bader, L. Bovard, M. Dumbser, K. Duru, F. Fambri, A.-A. Gabriel, J.-M. Gallard, S. Köppel, L. Krenz, L. Rannabauer, L. Rezzolla, P. Samfass, M. Tavelli, and T. Weinzierl, "Exahype: An engine for parallel dynamically adaptive simulations of wave problems," *Computer Physics Communications*, vol. 254, p. 107 251, Sep. 2020, ISSN: 0010-4655. DOI: 10.1016/j.cpc.2020.107251.

[2] T. Weinzierl, "The peano software—parallel, automaton-based, dynamically adaptive grid traversals," *ACM Transactions on Mathematical Software*, vol. 45, no. 2, p. 14, 2019.

[3] E. Gaburro and M. Dumbser, *A posteriori subcell finite volume limiter for general pnpm schemes: Applications from gasdynamics to relativistic magnetohydrodynamics*, 2020. arXiv: 2010.04853 [math.NA].

[4] V. Titarev and E. Toro, "Ader: Arbitrary high order godunov approach," *J. Sci. Comput.*, vol. 17, pp. 609–618, Dec. 2002. DOI: 10.1023/A:1015126814947.

[5] R. J. LeVeque, *Finite Volume Methods for Hyperbolic Problems* (Cambridge Texts in Applied Mathematics). Cambridge University Press, 2002.

[6] B. van Leer, "Towards the ultimate conservative difference scheme v. a second-order sequel to godunov's method," *Journal of Computational Physics*, vol. 32, pp. 101–136, Jul. 1979. DOI: 10.1016/0021-9991(79)90145-1.

[7] R. Courant, K. Friedrichs, and H. Lewy, "Über die partiellen differenzengleichungen der mathematischen physik," German, *Mathematische Annalen*, vol. 100, no. 1, pp. 32–74, 1928. DOI: 10.1007/BF01448839.

[8] M. Dumbser, C. Enaux, and E. F. Toro, "Finite volume schemes of very high order of accuracy for stiff hyperbolic balance laws," *Journal of Computational Physics*, vol. 227, no. 8, pp. 3971–4001, 2008, ISSN: 0021-9991. DOI: https://doi.org/10.1016/j.jcp.2007.12.005.

[9] I. Faille, F. Nataf, F. Willien, and S. Wolf, *Local time steps for a finite volume scheme*, 2008. arXiv: 0805.0550 [math.NA].

[10] A. Breuer and A. Heinecke, "Next-generation local time stepping for the ader-dg finite element method," in *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2022, pp. 402–413. DOI: 10.1109/IPDPS53621.2022.00046.

[11] L. D. S. Krenz, "A fully coupled model for petascale earthquake-tsunami and earthquake-sound simulations," Ph.D. dissertation, Technische Universität München, 2024.

[12] A. Breuer, A. Heinecke, and M. Bader, "Petascale local time stepping for the ader-dg finite element method," in *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2016, pp. 854–863. DOI: 10.1109/IPDPS.2016.109.

[13] T. Weinzierl and The Peano Development Team, *Peano: A framework for solvers on dynamically adaptive cartesian meshes (documentation)*, https://tum-i5.github.io/ExaHyPE-Documentation/index.html, Comprehensive documentation covering the spacetree-based architecture of Peano 4 and its integration with the ExaHyPE 2 engine for hyperbolic equation systems. Accessed: 2026-01-21, 2026.

[14] S. Wolf, M. Galis, C. Uphoff, A.-A. Gabriel, P. Moczo, D. Gregor, and M. Bader, "An efficient ader-dg local time stepping scheme for 3d hpc simulation of seismic waves in poroelastic media," *Journal of Computational Physics*, vol. 455, p. 110 886, Apr. 2022, ISSN: 0021-9991. DOI: 10.1016/j.jcp.2021.110886.

[15] Leibniz Supercomputing Centre, *Coolmuc-4 cluster at the leibniz supercomputing centre*, Accessed: 2026-02-08, https://doku.lrz.de/linux-cluster-10745672.html, Bayerische Akademie der Wissenschaften, Garching, Germany, 2024.

[16] M. Dumbser, D. S. Balsara, E. F. Toro, and C.-D. Munz, "A unified framework for the construction of one-step finite volume and discontinuous galerkin schemes on unstructured meshes," *Journal of Computational Physics*, vol. 227, no. 18, pp. 8209–8253, 2008, ISSN: 0021-9991. DOI: https://doi.org/10.1016/j.jcp.2008.05.025.

[17] D. L. George, "Augmented riemann solvers for the shallow water equations over variable topography with steady states and inundation," *Journal of Computational Physics*, vol. 227, no. 6, pp. 3089–3113, 2008, ISSN: 0021-9991. DOI: https://doi.org/10.1016/j.jcp.2007.10.027.